RAPPORT DE TRAVAUX PRATIQUES

TP 2 : Intégration de Notion et Git pour la Gestion de Projet

Application To-Do List

Module	Génie Logiciel
Encadrant	Pr. BE. ELBAGHAZAOUI
Réalisé par	Ikram El-MAZINI
Année universitaire	2025 / 2026

1. Introduction

Ce travail pratique a pour objectif de combiner deux outils modernes indispensables dans le développement logiciel : Notion, utilisé comme outil de documentation et de gestion de projet collaboratif, et Git avec GitHub, utilisés pour le contrôle de version du code source.

Le projet consiste à développer une application web de gestion de tâches (To-Do List) permettant d'ajouter, de supprimer et de marquer des tâches comme terminées. Chaque fonctionnalité est développée dans une branche Git dédiée, en suivant un workflow professionnel synchronisé avec un tableau Kanban dans Notion.

2. Objectifs du TP

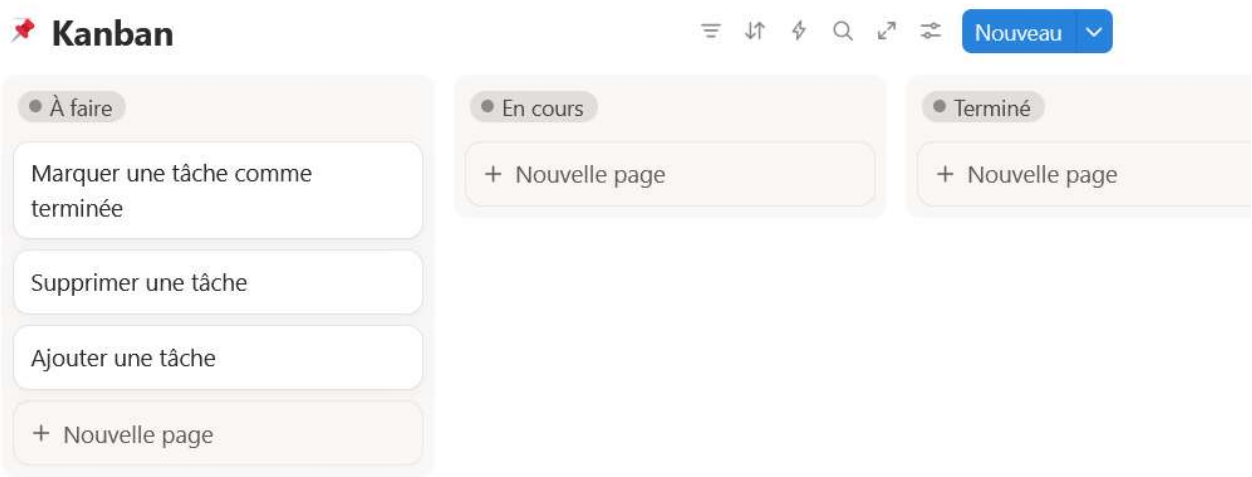
- Configurer un espace de travail collaboratif dans Notion avec un tableau Kanban.
- Initialiser un dépôt Git local et le publier sur GitHub avec les branches main et develop.
- Créer une branche Git dédiée pour chaque fonctionnalité de l'application.
- Synchroniser l'état du tableau Kanban Notion avec les branches Git.
- Documenter les problèmes rencontrés et leurs solutions dans Notion.
- Documenter une réunion de feedback récapitulative.

3. Étape 1 : Mise en place de Notion

Un compte Notion a été créé et une nouvelle page intitulée "Projet To-Do List" a été configurée. Cette page contient trois sections principales : le tableau Kanban, la documentation des exigences fonctionnelles et la section rapport.

3.1 Tableau Kanban

Le tableau Kanban est organisé en trois colonnes permettant de suivre l'état d'avancement de chaque fonctionnalité :



3.2 Documentation — Exigences fonctionnelles

Chaque fonctionnalité de l'application a été décrite dans la section Documentation de Notion :

Exigences fonctionnelles

- **Ajouter une tâche**

  L'utilisateur doit pouvoir créer une nouvelle tâche dans le Kanban.

  Chaque tâche doit être ajoutée avec un titre clair afin d'être facilement identifiable.

  Par défaut, une nouvelle tâche peut être placée dans la colonne **À faire**.

- **Supprimer une tâche**

  L'utilisateur doit pouvoir retirer une tâche qui n'est plus nécessaire.

  La suppression permet de garder le tableau organisé et d'éviter les éléments obsolètes.

  Cette action doit concerner uniquement la tâche sélectionnée.

- **Marquer une tâche comme terminée**

  L'utilisateur doit pouvoir déplacer une tâche vers la colonne **Terminé** lorsqu'elle est achevée.

  Cette fonctionnalité permet de suivre l'avancement du travail.

  Une tâche terminée doit rester visible afin de conserver l'historique des actions réalisées.

3.3 Section Rapport

Rapport

Cette section permet de noter les problèmes rencontrés pendant le projet ainsi que les solutions proposées ou appliquées.

- **Problème rencontré :**

  Décrire clairement le problème, son contexte et son impact sur le projet.
- **Solution proposée :**

  Expliquer la solution envisagée pour résoudre le problème ou limiter ses effets.
- **Solution appliquée :**

  Indiquer la solution réellement mise en place et préciser si elle a résolu le problème.
- **Remarques :**

  Ajouter toute information utile pour le suivi, les décisions prises ou les améliorations futures.

4. Étape 2 : Initialisation du projet avec Git

Un dépôt Git a été initialisé localement dans un nouveau dossier de projet. Un fichier README.md décrivant le projet a été ajouté. Deux branches principales ont été configurées : main (branche de production) et develop (branche de développement).

4.1 Commandes exécutées

Terminal / Git Bash

```
$ mkdir todo-list-app
$ cd todo-list-app
$ git init
$ echo "# Projet To-Do List — ENSA BM" > README.md
$ git add README.md
$ git commit -m "Initialisation du projet To-Do List"
$ git branch -M main
$ git checkout -b develop
$ git remote add origin https://github.com/ikram02-el/TPs_GenieLogiciel.git
$ git push -u origin main
$ git push origin develop
```

4.2 Structure des branches

Branche	Rôle
main	Branche principale — version stable de production
develop	Branche de développement — intègre toutes les fonctionnalités

> **Livrable : Lien GitHub — https://github.com/ikram02-el/TPs_GenieLogiciel**

5. Étape 3 : Suivi des tâches — Branches Git et Notion

Pour chaque fonctionnalité du tableau Kanban, une branche Git dédiée a été créée à partir de develop. Cette approche garantit l'isolation du développement de chaque fonctionnalité et facilite la revue de code.

5.1 Fonctionnalité 1 — Ajouter une tâche (feature/add-task)

Dans Notion, la carte "Ajouter une tâche" a été déplacée vers la colonne "En cours".

```
Terminal / Git Bash
$ git checkout develop
$ git checkout -b feature/add-task
# Développement de la fonction addTask() dans index.html
$ git add index.html
$ git commit -m "feat: Ajout de la fonctionnalité Ajouter une tâche"
$ git push origin feature/add-task
# Fusion dans develop
$ git checkout develop
$ git merge feature/add-task
$ git push origin develop
```

Dans Notion : la carte "Ajouter une tâche" a été déplacée vers la colonne "Terminé".

5.2 Fonctionnalité 2 — Supprimer une tâche (feature/delete-task)

Dans Notion, la carte "Supprimer une tâche" a été déplacée vers la colonne "En cours".

```
Terminal / Git Bash
$ git checkout develop
$ git checkout -b feature/delete-task
# Développement de la fonction deleteTask() dans index.html
$ git add index.html
$ git commit -m "feat: Ajout de la fonctionnalité Supprimer une tâche"
$ git push origin feature/delete-task
# Fusion dans develop
$ git checkout develop
$ git merge feature/delete-task
$ git push origin develop
```

Dans Notion : la carte "Supprimer une tâche" a été déplacée vers la colonne "Terminé".

5.3 Fonctionnalité 3 — Marquer comme terminée (feature/mark-done)

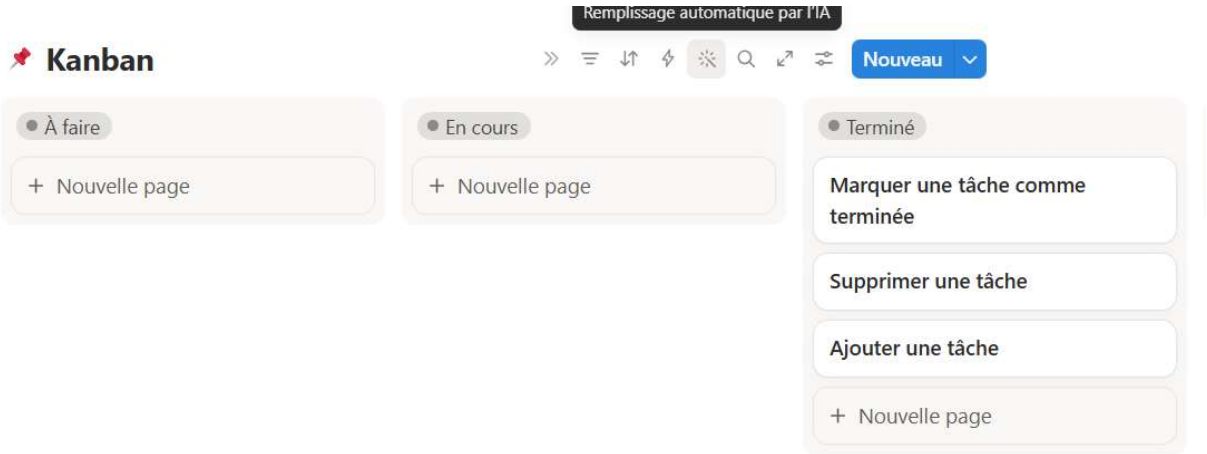
Dans Notion, la carte "Marquer une tâche comme terminée" a été déplacée vers la colonne "En cours".

```
Terminal / Git Bash
$ git checkout develop
$ git checkout -b feature/mark-done
# Développement de la fonction toggleTask() dans index.html
$ git add index.html
$ git commit -m "feat: Ajout de la fonctionnalité Marquer une tâche comme
terminée"
$ git push origin feature/mark-done
```

```
# Fusion dans develop
$ git checkout develop
$ git merge feature/mark-done
$ git push origin develop
```

Dans Notion : la carte "Marquer une tâche comme terminée" a été déplacée vers la colonne "Terminé".

5.4 État final du tableau Kanban



6. Étape 4 : Documentation des problèmes dans Notion

Une section "Problèmes rencontrés" a été ajoutée dans la page Notion. Pour chaque problème rencontré, nous avons documenté sa description, la solution apportée, et un lien vers le commit GitHub correspondant.

Problèmes rencontrés

Problème	Description + cause	Solution appliquée
Conflit de fusion Git	Lors de la fusion de `feature/delete-task` dans `develop`, un conflit est apparu car deux branches avaient modifié la même zone de `index.html`.	Ouverture de `index.html`, suppression des marqueurs de conflit (`<<<`, `===`, `>>>`), conservation de la version combinée, puis `git add` et `git commit`.
Branche mal nommée	La branche `feature/mark-done` avait été créée depuis `main` au lieu de `develop` par erreur.	Suppression de la branche incorrecte avec `git branch -d`, puis recréation depuis `develop` avec `git checkout develop && git checkout -b feature/mark-done`.
Oubli de git push	Après la fusion locale dans `develop`, la branche distante n'était pas mise à jour car `git push origin develop` avait été oublié.	Utilisation de `git push origin develop` pour synchroniser la branche distante. Ajout d'une vérification systématique avec `git status`.

7. Étape 5 : Collaboration et Feedback

Une réunion récapitulative a été documentée dans Notion afin de faire le bilan du projet et d'identifier les axes d'amélioration.

Réunion récapitulative

- **Progrès réalisés**
  - Application To-Do List entièrement développée avec les 3 fonctionnalités demandées.
  - Tableau Kanban Notion mis à jour en temps réel — les 3 cartes sont dans la colonne "Terminé".
  - 3 branches de fonctionnalités créées, développées, fusionnées dans `develop` et publiées sur GitHub.
  - Dépôt GitHub public avec un historique de commits clair et bien documenté.
  - Tous les problèmes rencontrés ont été documentés dans Notion avec leurs solutions.
- **Points à améliorer**
  - Ajouter la persistance des données avec `localStorage` pour conserver les tâches après rechargement de la page.
  - Mettre en place des Pull Requests sur GitHub pour simuler une revue de code collaborative avant chaque fusion.
  - Améliorer la documentation Notion avec des captures d'écran de chaque étape du développement.
  - Ajouter des tests automatiques pour valider chaque fonctionnalité avant la fusion dans `develop`.

8. Présentation du code source

L'application To-Do List a été développée en une seule page HTML avec CSS intégré et JavaScript pur (Vanilla JS). Ce choix garantit la simplicité du projet tout en couvrant toutes les fonctionnalités demandées.

8.1 Structure du fichier index.html

Section	Contenu
`<head>`	Métadonnées, titre de la page et styles CSS complets
`<body>`	En-tête, section d'ajout, statistiques, filtres, liste des tâches
`<script>`	Logique JavaScript : addTask(), deleteTask(), toggleTask(), render()

8.2 Fonctions JavaScript principales

Fonction	Branche Git	Description
`addTask()`	feature/add-task	Lit le champ de saisie, crée un objet tâche avec id, texte et état, l'ajoute au tableau tasks[] et appelle render()
`deleteTask(id)`	feature/delete-task	Filtre le tableau tasks[] pour retirer la tâche dont l'identifiant correspond, puis appelle render()
`toggleTask(id)`	feature/mark-done	Trouve la tâche par son id, inverse la valeur de completed (true/false), puis appelle render()
`clearCompleted()`	feature/delete-task	Supprime en une seule action toutes les tâches dont completed === true
`setFilter(filter)`	develop	Change le filtre actif (all / active / done) et met à jour l'affichage
`render()`	develop	Reconstruit entièrement la liste HTML, met à jour les compteurs et gère l'affichage du message "liste vide"

8.3 Extraits de code — Les 3 fonctionnalités

Fonctionnalité 1 : addTask()

Terminal / Git Bash

```
function addTask() {
  const input = document.getElementById('taskInput');
  const text = input.value.trim();
  if (!text) return;  // validation : champ non vide
  const task = {
    id: nextId++,
    text: text,
    completed: false,
    createdAt: new Date().toLocaleString('fr-FR')
```

```
  };
  tasks.push(task);
  input.value = '';
  render();
}
```

Fonctionnalité 2 : deleteTask()

```
Terminal / Git Bash
function deleteTask(id) {
  tasks = tasks.filter(t => t.id !== id);
  render();
}
```

Fonctionnalité 3 : toggleTask()

```
Terminal / Git Bash
function toggleTask(id) {
  const task = tasks.find(t => t.id === id);
  if (task) {
    task.completed = !task.completed;  // bascule l'état
    render();
  }
}
```

9. Résumé du workflow Git complet

N°	Commande Git	Action réalisée
1	`git init`	Initialisation du dépôt local
2	`git add / git commit`	Premier commit avec README.md
3	`git checkout -b develop`	Création de la branche develop
4	`git push origin main/develop`	Publication des branches principales sur GitHub
5	`git checkout -b feature/add-task`	Création de la branche pour Ajouter une tâche
6	`git merge feature/add-task`	Fusion dans develop après développement
7	`git checkout -b feature/delete-task`	Création de la branche pour Supprimer une tâche
8	`git merge feature/delete-task`	Fusion dans develop après développement
9	`git checkout -b feature/mark-done`	Création de la branche pour Marquer comme terminée
10	`git merge feature/mark-done`	Fusion dans develop après développement
11	`git push origin develop`	Publication de la version finale sur GitHub

10. Conclusion

Ce TP nous a permis de mettre en pratique un workflow de développement moderne et professionnel. En combinant Notion pour la gestion de projet et Git pour le contrôle de version, nous avons suivi une méthodologie complète : définition des tâches dans un tableau Kanban, développement par branches dédiées, documentation des problèmes, résolution des conflits et publication sur GitHub.

L'application To-Do List développée implémente toutes les fonctionnalités demandées (ajout, suppression, marquage des tâches) avec une interface utilisateur claire et intuitive. Chaque fonctionnalité a été développée dans sa propre branche Git, en parfaite synchronisation avec l'état du tableau Kanban dans Notion.

Ce TP illustre l'importance de combiner un outil de gestion de projet (Notion) avec un outil de contrôle de version (Git) pour mener à bien un projet logiciel de manière organisée et traçable.